Steering the Linux Scheduler with Koopman Burst Prediction under EEVDF

Saarthak Manocha
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
saarthakmanocha@gmail.com

***Abstract*—Shortest-job-first scheduling needs each task's next CPU burst before it runs. We predict bursts with a Koopman operator, fitted by dynamic mode decomposition on log-scale delay coordinates, and feed the predictions to the unmodified Linux EEVDF scheduler as per-task slice requests. Every measurement is validated against the kernel's own accounting (CPU time within 0.93%, wait within 0.22%, 105 processes). On real programs (Redis, SQLite, Node.js, ffmpeg) the predictor beats the textbook exponential average in 32 of 36 threads, with 29% lower error. We show it is numerically a log-scale autoregressive model, and that neither nonlinear observables, ARIMA nor a neural network beat that model. Closing the loop with an eBPF observer that costs 0.33% of one core, predicted slices cut short-request response time by 45% with no loss of throughput, and the Koopman predictor beats the exponential average in 19 of 24 paired runs. We show why slices need a margin above the predicted burst: an exact slice gives no gain, 1.5 times the burst gives 61%. With real servers sharing a core with CPU hogs, Redis requests became 31–37% faster and SQLite point queries 31% faster, with tail latency unchanged. We release the capture kit, data, controller, a sched_ext policy for bare-metal tests and a scheduler visualizer.**

***Index Terms*—Koopman operator, dynamic mode decomposition, CPU scheduling, EEVDF, shortest job first, eBPF, kernel tracing, conformal prediction**

I. Introduction

Shortest-job-first (SJF) scheduling minimizes mean waiting time, but it needs the length of each task's next CPU burst before the burst runs. Textbooks teach an estimate, the exponential average $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$ [1], and production schedulers avoid prediction: Linux's fair class has, since kernel 6.6, picked the task with the earliest eligible virtual deadline (EEVDF) [2], [3]. Since kernel 6.12, however, EEVDF lets each task request its own time slice, and a shorter slice means an earlier deadline [4], [5]. That turns burst prediction from a textbook exercise into a lever on a production scheduler: if a task's next burst is known, its slice can be set to match.

We ask three questions with real kernel data. Can bursts be predicted better than with the textbook estimator? Does a better prediction make the real scheduler faster? And what, specifically, does a Koopman view add? Koopman operator theory [6]–[8] represents a nonlinear system by a linear operator on functions of its state; dynamic mode decomposition (DMD) and its extension EDMD [9], [10] estimate it from data, and its eigenvalues describe how patterns decay. Applied to delay coordinates (Hankel DMD) it is closely related to autoregressive models [11], [12]; we test whether, for CPU bursts, it is more than that.

Contributions.

1) A validated measurement pipeline for scheduler dynamics: capture without perturbation, ground truth from `schedstat`, and two documented measurement hazards (Section IV).
2) Burst prediction on synthetic workloads and four real programs against autoregressive, ARIMA, neural and nonlinear-Koopman baselines, with prediction intervals and cost (Section V).
3) A closed-loop controller that steers unmodified EEVDF through slice requests, first by polling and then with an eBPF observer, evaluated in 264 paired runs including real servers, with a model of why slices need a margin (Sections III and VI).
4) Multi-core analysis, a machine-level change detector, a trace visualizer, and a sched_ext policy for bare-metal tests (Sections VII–IX).

II. Background and Related Work

Koopman operators and DMD. For a map $x_{t+1} = F(x_t)$ the Koopman operator advances observables, $(\mathcal{K}g)(x) = g(F(x))$, linearly even when F is not [6]. EDMD approximates it by the matrix $\mathbf{K}$ minimizing $\|\Psi_Y - \Psi_X \mathbf{K}\|_F$ over snapshot pairs of a dictionary ψ [10]; with $\psi(x) = x$ it is DMD [9], [13], and it converges to the projection of $\mathcal{K}$ onto the dictionary span [14]. Delay coordinates are justified by Takens' theorem [11], [15]; linear models on them include AR models and Hankel DMD as special cases [12]. Noise biases eigenvalues towards zero [16].

Learning in schedulers. Negi and Kumar predicted CPU bursts from process attributes with decision trees and set Linux 2.4 time slices from them, reducing turnaround by 1.4–5.8% [17]. Size-based scheduling helps heavy-tailed workloads [18]. sched_ext (6.12) lets BPF programs implement scheduling policies [19]; recent work uses a learned classifier to switch between whole sched_ext policies [20]. EEVDF slice requests were added so that latency-sensitive tasks can be served sooner, at some cost to throughput if slices are too short [4]. We know of no study that sets them from predicted bursts.

TABLE I
PLATFORMS AND DATA

Platform	Kernel	Capture	Runs
Lab, 28 CPUs, bare metal	7.0	perf	1 (75 s)
Intel Xeon VM, 2 vCPUs (KVM)	6.18	tracefs	12 + 24 + 3
same VM, closed loop	6.18	counters, eBPF	264
ARM VM, 4 vCPUs (Apple host)	6.8	counters	9 + 24

III. METHOD

Predictor. For each task, the state is its last three bursts on a log scale, $z_t = (\log b_t, \log b_{t-1}, \log b_{t-2})$, standardized with training statistics, and one operator is fitted by least squares, $\mathbf{K} = \arg\min_A \sum_t \|z_{t+1} - z_t A\|^2$. The predicted burst is the first coordinate of $z_t\mathbf{K}$, mapped back. Because the dictionary is linear in delay coordinates, this is numerically a log-scale AR(3) model: the two give the same forecasts (median of the largest log-space difference 10^{-3} over 96 tasks). We call it log-EDMD and also test nonlinear dictionaries: squares and products of the lags (*poly*) and 64 random Fourier features (*RFF*, approximate kernel EDMD). In the controller the operator is updated after every burst by recursive least squares with forgetting factor 0.995. The eigenvalues of $\mathbf{K}$ describe each task's burst dynamics.

From prediction to slice. The slice requested for a task is $s = c\,\hat{b}$, clamped between the kernel minimum (0.1 ms) and the default slice (1.4 ms here), where $\hat{b}$ is the predicted burst and c a margin. All tasks have equal weight.

Why a margin. Under EEVDF a woken task gets virtual deadline $v + s$. A smaller s makes it more likely to preempt the running task at once (allowed when its slice is shorter and its deadline earlier) and to be picked next, so its wait $W(s)$ grows with s. But if its actual CPU demand b' (the burst plus system overhead) exceeds s, its deadline is renewed while it still runs and it loses its place to tasks with earlier deadlines, an extra wait Δ of the order of their slices. So

$$\mathbb{E}[R \mid s] \approx W(s) + b' + \Delta \Pr(b' > s), \tag{1}$$

and with $s = c\hat{b}$ and log-normal prediction error of spread σ, $\Pr(b' > s) \approx 1 - \Phi(\log c/\sigma)$ when the overhead is small. Two consequences follow. Even a perfect predictor needs $c > 1$, because $b' > b$. And at a given margin, a predictor with smaller σ overruns less, so differences between predictors should show at small c and vanish at large c, where slices saturate at the default. Section VI tests both.

IV. MEASUREMENT AND VALIDATION

Platforms and workloads (Table I). All kernels run EEVDF. Synthetic workloads are a CPU-bound loop, an I/O-bound loop that writes 10 kB and sleeps 50 ms, and a *phased* positive control alternating every 2 s between 1 ms and 20 ms bursts. The lab trace (*Trace 3*) ran three CPU-bound and three I/O-bound jobs; VM captures ran these plus two phased jobs, pinned to 1 or 2 cores, overloaded, or with a load drop, three times each. Real programs and the closed-loop workloads are described in Sections V and VI.

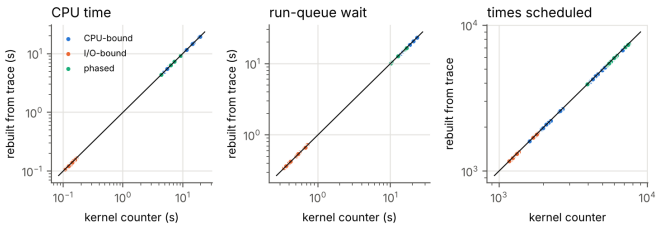


Fig. 1. Every reconstructed process against the kernel's own counters (105 processes, 12 captures).

Reconstruction and ground truth. From `sched_switch`, `sched_wakeup` and `sched_migrate_task` events we rebuild, per task, every interval on a CPU (with its state at switch-out), every interval waiting in a run queue, and every migration. A burst is the CPU time between two blocking events. Each workload saves the kernel's own `/proc/self/schedstat` and context-switch counts before it exits. Over 105 processes in 12 VM captures (Fig. 1), reconstructed CPU time differs from the kernel's by 0.93% (median; max 3.45%), wait time by 0.22% (max 2.07%), and the number of times scheduled by a median of 4, with no lost events.

Hazard 1: the observer effect. A first capture tool streamed events while recording. Each event woke the reader, which preempted the workload and produced further events: about 50,000 context switches per second on any core with spare capacity, traces 20–30 times larger, and counts off by up to 9%. Recording into a large kernel buffer and reading it after the run removed it.

Hazard 2: missing idle exits on a VM. On the Intel VM's kernel, part of the `sched_switch` events for a CPU leaving idle are never emitted (tracefs and perf agree; the ring buffer reports no loss): a task that slept 201 times was seen switching in only 1 to 176 times across seven self-tests. The lab machine does not show it. Our tools count unmatched switch-outs and keep VM cores busy, with workload or with one lowest-priority (`SCHED_IDLE`) spinner per core, after which counts match the kernel.

V. BURST PREDICTION

Protocol. Every method predicts a burst from earlier bursts of the same task; the last 20% of each task is the test set. Methods: mean, last value, the textbook exponential average ($\alpha = 0.5$, EWMA) and with tuned α, log-EDMD, poly- and RFF-EDMD, log-AR with order 1–10 chosen by AIC (log-AR(p^*)), ARIMA chosen by AIC, a two-layer neural network on six log lags, and gradient boosting on three raw lags. Intervals are moving-block bootstrap [21]; sign tests are Holm-corrected [22].

Synthetic workloads. On the idle 28-core lab machine, I/O bursts tripled on the same cores when the CPU-bound jobs exited (Fig. 2), a level shift that the exponential average follows best; the I/O operators' dominant eigenvalues, 0.88–0.97, mark it. Under contention on the VM, the positive control's

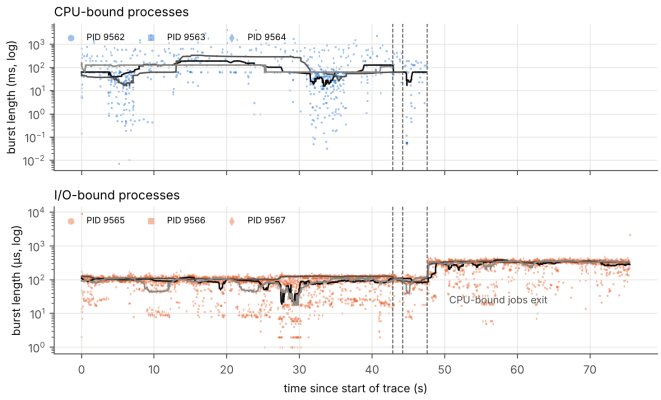


Fig. 2. Trace 3 (bare metal, 28 cores): every burst over time. I/O bursts lengthen about threefold when the CPU-bound jobs exit (dashed lines).

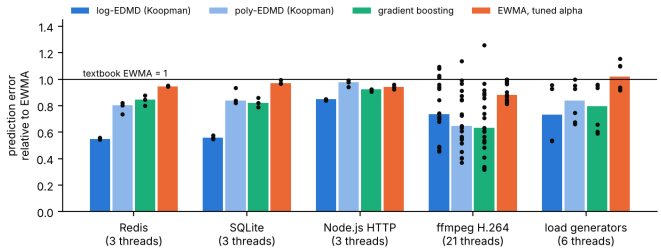


Fig. 3. Real programs: burst-prediction error relative to the textbook exponential average (bars: median; dots: threads; three captures).

structured bursts are captured by every method that uses recent bursts (90–96% lower error than the mean), best by tuned EWMA; log-EDMD is 5–23% worse there. On contended I/O bursts the order reverses: log-EDMD beats EWMA in 33 of 33 processes (median error ratio 0.86, $p_{\text{Holm}} = 2\times10^{-9}$), because the exponential average overreacts to single long bursts.

Real programs. We traced, together on both VM cores for three 60 s captures, a Redis 7.0 server driven by a paced client, a Node.js 22 HTTP server with phases of light and heavy JSON requests, SQL queries of mixed size against SQLite 3.45, real-time H.264 encoding with ffmpeg (all threads), and the load generators, with a gcc build and xz compression as background load. Over the 36 threads with at least 60 bursts, CPU time agrees with `schedstat` within 1.8% and wait within 1.5% (medians). Log-EDMD beats EWMA in 32 of 36 threads, with median error ratio 0.71 ($p_{\text{Holm}} = 10^{-5}$; Fig. 3); the gain is largest for Redis and SQLite (0.55 and 0.56), whose short request bursts are interrupted by occasional long ones.

Is it more than an AR model? Table II answers no. Every model that uses recent bursts beats the textbook estimator, all by similar amounts. The best single family is a log-scale linear model with enough delays: log-AR with AIC-chosen order (most often 10) is better than log-EDMD with three delays in 75 of 96 tasks, though only by 1.5% at the median; this is Hankel DMD with more delay coordinates. Nonlinear dictionaries, ARIMA, the neural network and gradient boosting do not beat it (Holm $p \geq 0.37$ for the two nonlinear Koopman variants). What predicts CPU bursts is the log scale and the delay depth, not nonlinearity. The Koopman view contributes the eigenvalues: fitted on windows of 150 bursts, they alone identify which of the five programs a window came from with 76% balanced accuracy (chance 20%; SQLite's are all real, the load generators' slowest mode is 0.87), though simple burst statistics do better (99.8%).

TABLE II
IS IT MORE THAN AN AR MODEL? MEDIAN ERROR RATIO AND NUMBER OF TASKS WHERE THE METHOD IS BETTER, AGAINST TWO REFERENCES (96 TASKS, 36 OF THEM THREADS OF REAL PROGRAMS)

	vs EWMA		vs log-AR(p^*)	
Method	all 96	real 36	all 96	real 36
log-EDMD (3 delays)	0.88, 65	0.71, 32	1.02, 21	1.01, 8
log-AR(p^*), AIC	0.79, 66	0.70, 32	–	–
poly-EDMD	0.86, 68	0.71, 35	1.01, 41	1.00, 20
RFF-EDMD (kernel)	0.85, 60	0.74, 28	1.00, 46	1.00, 20
ARIMA, AIC	0.85, 66	0.71, 33	1.02, 31	1.00, 15
neural network	0.84, 66	0.74, 32	1.02, 34	1.00, 18
gradient boosting	0.87, 68	0.79, 34	1.03, 33	1.04, 14

Uncertainty and cost. Split conformal prediction [23], [24] on log-errors gave the target 90% coverage (median 0.90 for log-EDMD, 0.91 for EWMA on I/O tasks), with narrower intervals for log-EDMD in 34 of 60 tasks (median width ratio 0.84). In C, one prediction takes 5.6 ns and a recursive-least-squares update 29 ns, less than a context switch.

VI. Closing the Loop: Steering EEVDF

Workload. Request tasks (C) wake on a period of 15–25 ms and do one CPU burst each, drawn from a two-regime log-autoregressive process (short bursts near 0.3 ms, long ones near 4 ms, switching with probability 0.05), logging deadline, wake-up and finish; CPU hogs spin and count the CPU time they get. Every repetition runs every policy with the same seeds in shuffled order. The tasks' own CPU accounting agrees with `schedstat` within about 1% (median). The main metric is the mean response time of short requests (under 1 ms of CPU), from deadline to finish.

Polling agent. A first agent polled `/proc` every 10 ms. On one CPU (three hogs, four request tasks, eight repetitions) and two CPUs (six and eight, six repetitions; 635,531 requests in all), both predictors cut short-request response in every paired repetition, by 28–31% (Wilcoxon $p = 0.008$ and 0.03, the smallest attainable), at a 3–7% throughput cost that equals the agent's own CPU time. Hand-set 0.1 ms slices for the request tasks cut wake-up latency but not response (-3%): every burst is chopped into 0.1 ms pieces, with 7–8 times more context switches.

eBPF agent. A BPF program on the `sched_switch` tracepoint reports every burst the moment it ends, and a C agent updates the task's predictor and sets its slice. It uses 0.33% of one core, against 3% for the polling agent, and CPU hogs lose no measurable throughput (within $\pm1\%$ of stock). Fig. 4 (left) sweeps the margin, and both predictions of the

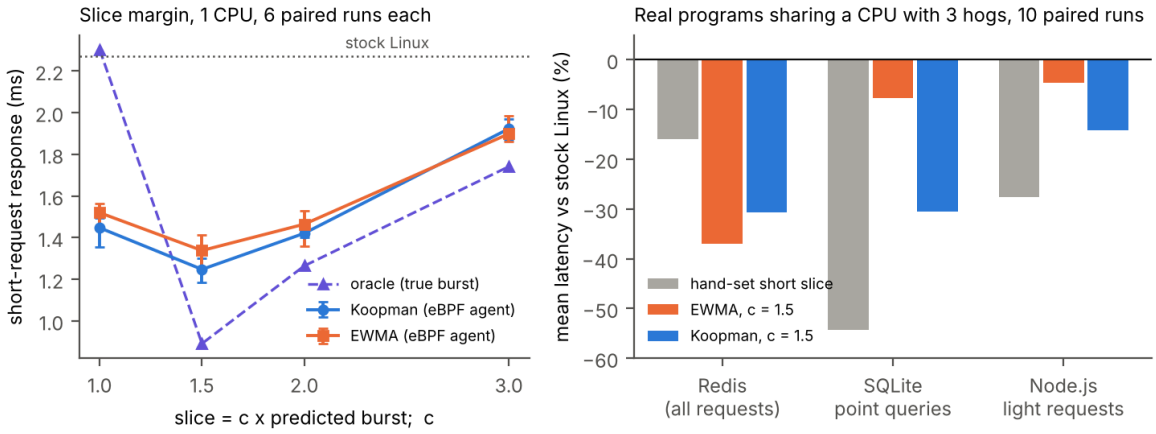


Fig. 4. Left: short-request response against the slice margin c (eBPF agent, one CPU with three hogs; bars: range over six paired runs); the oracle knows each burst exactly. Right: change in mean latency of real programs sharing one CPU with three hogs (ten paired runs), against stock EEVDF.

model hold. The oracle with $c = 1$ is no better than stock Linux (+1.5%), while $c = 1.5$ gives -61% and larger margins lose ground (-44% at 2, -23% at 3). The predictors follow the same curve, with the best margin also at 1.5: Koopman -45% (1.25 against 2.27 ms), EWMA -41%, and mean response over *all* requests -5 to -6%. Koopman beats EWMA in 6 of 6 paired runs at $c = 1$ and at $c = 1.5$ ($p = 0.031$ each), 5 of 6 at 2, and 2 of 6 at 3; over all margins in 19 of 24 ($p = 0.0035$). Its burst predictions are 9% more accurate (mean absolute error 0.77 against 0.84 ms; log-error spread 0.49 against 0.58).

Real programs. A Redis server, a Node.js server and a SQLite worker shared one CPU with three hogs, with their clients on the other CPU timing every request end to end; the agent controlled the three server threads (ten paired repetitions; Fig. 4, right). With predicted slices at $c = 1.5$, Redis request latency fell by 31% (Koopman) and 37% (EWMA), SQLite point queries by 31% and 8%, and Node.js light requests by 14% ($p = 0.01$) and 5% (not significant); the Koopman changes for Redis and SQLite held in 10 of 10 pairs ($p = 0.002$). Koopman was 25% better than EWMA on SQLite point queries (10 of 10) and 10% worse on Redis (1 of 10). With either predictor at $c = 1.5$, the mean over all Node.js and SQLite requests, heavy requests, and the 99th percentiles of all three programs changed by less than 4% (not significant), and hogs lost 1% of CPU time. Hand-set minimum slices for the servers did best for SQLite point queries (-54%) and Node.js light requests (-28%) but worst for Redis (-16%), and require knowing which tasks are latency-sensitive.

VII. Multi-Core Scheduling

On the lab machine, processes waited a median 16 μs for a CPU: no queue formed, so no pick order could matter. On the ARM VM, from kernel counters alone, CPU-bound jobs spent 76% of their time waiting on one core, 50% on two and 1% on four; the Intel VM traces agree. A trace-driven simulator replaying each traced task's real bursts and blocking times under FCFS, round robin, non-preemptive SJF and SRTF with

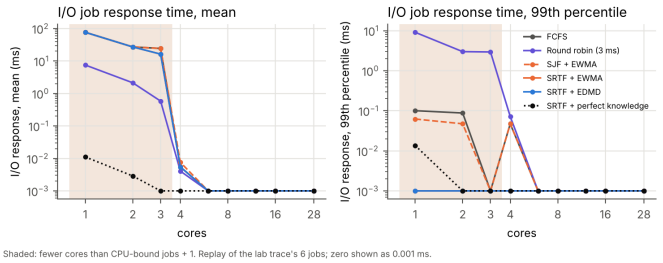


Fig. 5. Simulated I/O response time versus core count, replaying the lab trace's six jobs.

each predictor (Fig. 5) shows I/O response time zero for every policy once cores outnumber CPU-bound jobs. Below that, non-preemptive SJF behaves like FCFS whatever the predictor (10.2 against 10.4 ms at two cores), because an I/O job must wait for the CPU-bound job holding the core; SRTF with any predictor brings it to 0.011 ms. Measured EEVDF on one core gave interactive jobs 1.87 ms against 1.5–1.9 ms for simulated SRTF with predictors. Prediction pays only where tasks queue, and only through preemption, which is what slice requests provide.

VIII. Koopman View of the Whole Machine

The same estimator applied to a machine-wide state (busy fraction and queued tasks per core, CPU share per job class, every 10 ms) detects load changes from its one-step residual. In three VM runs where four of six CPU-bound jobs stop while every core stays busy, it raised an alarm 77–94 ms after the change, with no false alarm before it or in nine runs without a change; utilization, persistence and distance-from-normal monitors detected none (Fig. 6). A live version reading only `/proc/stat`, unprivileged and at 0.5–1.1% of one core, detected 29 of 36 scripted changes on two VMs with a median delay of 0.18 s, but a static distance-from-normal score on the same three variables was as good or better in 51 of 54 settings:

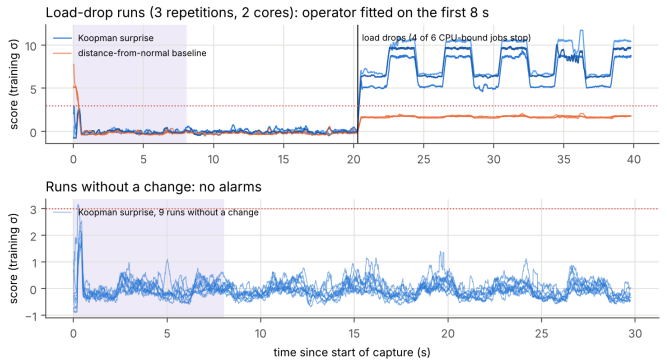


Fig. 6. Trace-based detector on the VM. Top: three runs with a load drop at 20 s. Bottom: nine runs without a change. Shaded: training.

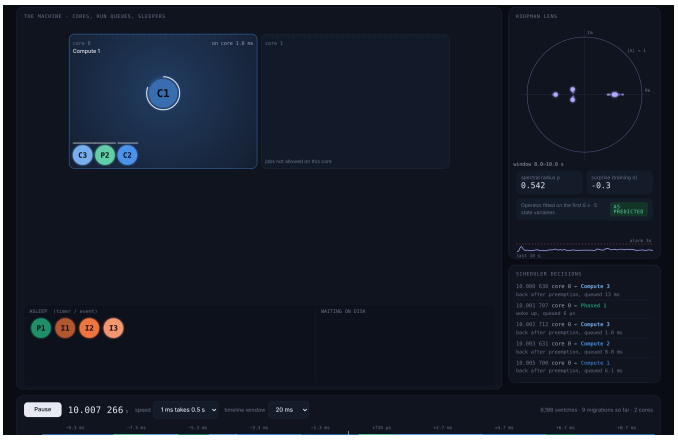


Fig. 7. Scheduler Observatory replaying one core shared by eight jobs: running job, queued jobs with wait bars, sleepers, Koopman eigenvalues and decision log.

on low-dimensional counters, watching the variables jointly matters more than modelling their dynamics.

IX. Making the Scheduler Visible

The Scheduler Observatory (Fig. 7) replays validated traces in a browser: each core is a tile lit in the colour of the task it runs, queued tasks wait under their core with growing wait bars, sleeping tasks rest in a pool, migrations are drawn as arcs, a log narrates each decision, and a Koopman panel shows the eigenvalues of a sliding-window operator and the live residual.

X. Beyond the Operating System

The same estimator has also been applied above the operating system; we summarize these as case studies. **Network traffic** [25]: 16.2 million CSE-CIC-IDS2018 flow records [26] were binned into 30 s windows with an 11-dimensional state; one 11×11 operator lowered forecast mean squared error against persistence on a held-out day by 25.9% at 30 s, 29.1% at 60 s and 18.9% at 15 min, and by 20.7% on UNSW-NB15 [27]; its slowest mode ($\lambda = 0.995$) has a half-life of 71 min. **Markets**: EIGENWORLD [28] fits a Koopman operator on diffusion-map coordinates [29] of seven daily

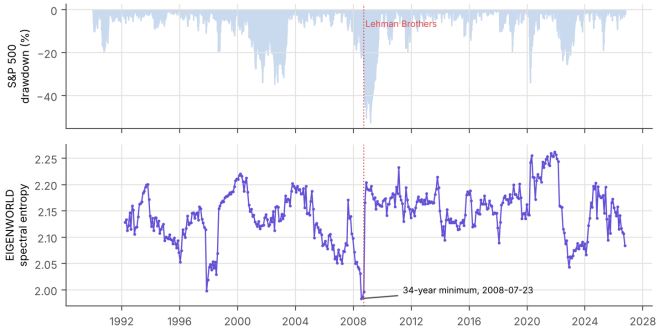


Fig. 8. S&P 500 drawdown and EIGENWORLD's spectral entropy (seed 0), 1992–2026; the 34-year minimum falls in July 2008.

TABLE III
Summary of Results

Task	Result	Compared with
Real-program bursts	29% lower error, 32/36 threads	textbook EWMA
Contended I/O bursts	14% lower error, 33/33	textbook EWMA
Nonlinear observables	no gain	log-AR(p^*)
Steering EEVDF (eBPF)	short requests 45% faster, no throughput loss	stock Linux
Koopman vs EWMA in loop	better in 19/24 runs	EWMA predictor
Real servers	Redis 31–37%, SQLite points 31% faster; tails unchanged	stock Linux
Load change, traces	3/3, 77–94 ms	baselines 0/3
Load change, live	29/36, 0.18 s	static score as good

S&P 500 features. Rerun under 11 fixed seeds, 10 reproduce its published minimum of spectral entropy on 2008-08-27, 19 days before the Lehman Brothers bankruptcy, and on 1992–2026 the lowest entropy in 34 years falls in July–August 2008 for every seed (Fig. 8).

XI. Discussion

Table III collects the results. For burst prediction, the Koopman framing earns its place through the log scale, the delay coordinates and the eigenvalues, not through nonlinear observables: on scheduler data, Hankel DMD and a log-scale autoregressive model are the same predictor, and richer models add nothing. Its practical value is in the loop. Predicted bursts, turned into EEVDF slice requests with a margin of about 1.5, make short requests 40–45% faster on a contended core, cost nothing in throughput when observed through eBPF, and speed up real servers' requests by about a third. How the forecast becomes a slice matters more than which forecaster is used: the wrong margin erases the gain even for a perfect predictor. Within that, a more accurate predictor helps where slices are tight, as the model predicts. The real-server results also show the limits: tails and heavy requests are unchanged, and for some programs a hand-set slice is better, if one knows which tasks to set.

Threats to validity. The closed-loop and real-program experiments ran on a 2-vCPU VM with a 250 Hz tick and no kernel preemption; hypervisor scheduling adds noise, and the best margin may differ elsewhere. The closed-loop request workload is synthetic; the real-program loop covers three servers on one contended core. Only one bare-metal trace exists, without contention. The multi-core simulator is idealized; an EEVDF simulator we built did not reproduce the measured closed-loop effects and is not used. Network numbers come from the original study; the market result reproduces one published event.

XII. Conclusion and Future Work

A Koopman operator on log-scale delay coordinates predicts CPU bursts better than the textbook estimator, by 29% on real programs; it is an autoregressive model in disguise, and nonlinear observables do not improve it. Used to set EEVDF slice requests with a margin, it makes short requests up to 45% faster on the stock Linux scheduler at negligible cost, and real server requests about a third faster. Remaining work needs bare metal: the closed loop on 8–28 cores, the real-program loop, contended scaling captures, and the frequency test that would explain the threefold I/O slowdown in Trace 3. For that test we provide a sched_ext policy (`scx_koopman`) that orders tasks by virtual time plus predicted burst; it compiles but has not yet been run.

Data and Code Availability

Capture kit, eBPF observer, controller, sched_ext policy, parsers, validation, simulators, all analyses and data (72 validated captures and counter runs, 264 closed-loop runs), the EIGENWORLD reproduction and the visualizer are at https://github.com/SaarthakManocha/Koopman-Operator-Theory-Research; its README lists the script behind each table and figure.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[2] I. Stoica and H. Abdel-Wahab, "Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation," Old Dominion University, Tech. Rep. TR-95-22, 1995.

[3] J. Corbet, "An EEVDF CPU scheduler for Linux," LWN.net, Mar. 2023, https://lwn.net/Articles/925371/.

[4] ——, "Completing the EEVDF scheduler," LWN.net, Apr. 2024, https://lwn.net/Articles/969062/, 2024.

[5] The Linux kernel documentation, "EEVDF scheduler," https://docs.kernel.org/scheduler/sched-eevdf.html, 2024.

[6] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Natl. Acad. Sci. USA*, vol. 17, no. 5, pp. 315–318, 1931.

[7] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dyn.*, vol. 41, no. 1–3, pp. 309–325, 2005.

[8] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Rev.*, vol. 64, no. 2, pp. 229–340, 2022.

[9] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *J. Fluid Mech.*, vol. 656, pp. 5–28, 2010.

[10] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.

[11] H. Arbabi and I. Mezić, "Ergodic theory, dynamic mode decomposition, and computation of spectral properties of the Koopman operator," *SIAM J. Appl. Dyn. Syst.*, vol. 16, no. 4, pp. 2096–2126, 2017.

[12] S. Pan and K. Duraisamy, "On the structure of time-delay embedding in linear models of non-linear dynamical systems," *Chaos*, vol. 30, no. 7, p. 073135, 2020.

[13] J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton, and J. N. Kutz, "On dynamic mode decomposition: Theory and applications," *J. Comput. Dyn.*, vol. 1, no. 2, pp. 391–421, 2014.

[14] M. Korda and I. Mezić, "On convergence of extended dynamic mode decomposition to the Koopman operator," *J. Nonlinear Sci.*, vol. 28, no. 2, pp. 687–710, 2018.

[15] F. Takens, "Detecting strange attractors in turbulence," in *Dynamical Systems and Turbulence, Warwick 1980*, ser. Lecture Notes in Mathematics. Springer, 1981, vol. 898, pp. 366–381.

[16] S. T. M. Dawson, M. S. Hemati, M. O. Williams, and C. W. Rowley, "Characterizing and correcting for the effect of sensor noise in the dynamic mode decomposition," *Exp. Fluids*, vol. 57, no. 3, p. 42, 2016.

[17] A. Negi and P. K. Kumar, "Applying machine learning techniques to improve Linux process scheduling," in *Proc. IEEE Region 10 Conf. (TENCON)*, 2005.

[18] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, "Size-based scheduling to improve web performance," *ACM Trans. Comput. Syst.*, vol. 21, no. 2, pp. 207–233, 2003.

[19] The Linux kernel documentation, "Extensible scheduler class (sched_ext)," https://docs.kernel.org/scheduler/sched-ext.html, 2024.

[20] X. Wang, S. Jia, J. Cao, Z. Huang, and M. Song, "Mixture-of-schedulers: An adaptive scheduling agent as a learned router for expert policies," arXiv:2511.11628, 2025.

[21] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *Ann. Statist.*, vol. 17, no. 3, pp. 1217–1241, 1989.

[22] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[23] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.

[24] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Found. Trends Mach. Learn.*, vol. 16, no. 4, pp. 494–591, 2023.

[25] S. Manocha, R. R. Lingala, C. S. M. Sadhana, S. Kedia, C. S. M. Sahithi, and P. Vemuri, "Koopman operator theory for network traffic dynamics: An inspectable linear world model for attack forecasting," 2026, manuscript, Chaitanya Bharathi Institute of Technology.

[26] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116.

[27] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.

[28] S. Manocha, "EIGENWORLD: Causal-topological intelligence for complex systems," Source code, https://github.com/SaarthakManocha/eigenworld, 2026.

[29] R. R. Coifman and S. Lafon, "Diffusion maps," *Appl. Comput. Harmon. Anal.*, vol. 21, no. 1, pp. 5–30, 2006.